nature machine intelligence

Article

https://doi.org/10.1038/s42256-025-01137-0

Densing law of LLMs

Received: 11 January 2025

Accepted: 22 September 2025

Published online: 6 November 2025

Check for updates

Chaojun Xiao[1,2], Jie Cai[3], Weilin Zhao[1,2], Biyuan Lin[3], Guoyang Zeng[3], Jie Zhou[3], Zhi Zheng[3], Xu Han[1,2] ✉, Zhiyuan Liu[1,2] ✉ & Maosong Sun[1,2] ✉

Large language models (LLMs) have emerged as a milestone in artificial intelligence. The scaling law indicates that the performance of LLMs can continually improve as the model size increases, which poses challenges for training and deployment. Despite numerous efforts to improve LLM efficiency, there is no general consensus on development trends and evaluation metrics for efficiency of LLMs with different scales. To address this tension between model performance and efficiency, we introduce the concept of capability density as a metric to evaluate the quality of the LLMs and describe the trend of LLMs in terms of both effectiveness and efficiency. Intuitively, capability density can be understood as the capability contained within each unit of model parameters. Capability density provides a unified framework for assessing both model performance and efficiency. Here we show an empirical observation, called the 'densing law', that the capability density of LLMs grows exponentially over time. More specifically, using widely used benchmarks for evaluation, the maximum capability density of open-source LLMs doubles approximately every 3.5 months. This reveals that both parameter requirements and inference costs of LLMs for achieving equivalent performance decrease exponentially, offering insights for efficient LLM development strategies.

In recent years, large language models (LLMs) have garnered widespread attention in the field of artificial intelligence (AI), demonstrating remarkable improvements across various tasks[1–5]. The scaling law for LLMs further reveals that model performance continues to improve as model parameters and training data increase[6–8]. This law has led to the development of LLMs with hundreds of billions of parameters, for example GPT-3 175B[9], PaLM 540B[10] and Llama-3.1-405B[11], and all of these large models have shown exceptional capabilities in a wider range of applications.

With the advancement of LLMs, enhancing inference efficiency has become increasingly urgent: (1) As LLMs are deployed in an expanding array of scenarios, inference costs have surpassed training costs, becoming the main bottleneck in practical applications[12–14]; (2) there is a growing need to deploy LLMs on resource-constrained end-side devices (for example, personal computers and smartphones) to serve as personal assistants, which requires models to be more efficient and compact[15–19]; and (3) the inference scaling law indicates that allowing LLMs to generate more tokens for 'thinking' during the inference stage is crucial for improving performance in complex reasoning tasks[20–22], further increasing the demand for efficient inference. To address these challenges, many efforts have been devoted to developing efficient LLMs to reduce inference overhead, for example OpenAI's mini-series models, GPT-4o-mini[14] and o1/o3-mini[23], Google's flash-series models, Gemini-2.0/2.5-Flash[24] and Apple's Apple Intelligence[15].

Given these two seemingly contradictory paths—scaling up LLMs for performance versus scaling down LLMs for efficiency—natural questions arise. Can we quantitatively evaluate the quality of LLMs with different scales? Is there a law that reflects the efficiency trend in LLMs, like the scaling law does for model performance?

To this end, we introduce the concept of capability density, which serves as a metric for evaluating and comparing the training quality of LLMs on various scales. Intuitively, capability density refers to the model capability per unit of parameters. However, unlike mass density, the relationship between capability and the number of parameters is

[1]Department of Computer Science and Technology, Institute for AI, Tsinghua University, Beijing, China. [2]Beijing National Research Center for Information Science and Technology, Beijing, China. [3]OpenBMB, Beijing, China. ✉e-mail: han-xu@tsinghua.edu.cn; liuzy@tsinghua.edu.cn; sms@tsinghua.edu.cn

not linear, so we cannot simply calculate the capability density value by dividing the model performance by the parameter size. Thus, in this paper, we design a method to assess the relative capability density. For ease of explanation, in this work, we use 'density' to refer to '(relative) capability density'. Specifically, we use a reference model and then estimate its scaling curve between the performance on downstream tasks and parameter sizes. Based on the scaling curve, for any given model, we calculate its effective parameter size, which is the number of parameters the reference model would need to achieve equivalent performance. The density of an LLM relative to the reference model is then defined as the ratio of its effective parameter size to its actual parameter size. By introducing the concept of model density, we aim to more accurately measure model quality and enable comparisons between models of different scales. This evaluation method has the potential to provide insights into the future direction of LLM development, helping researchers to find the optimal balance between performance and efficiency.

After establishing the concept of LLM capability density, we conduct a comprehensive analysis of the 51 open-source pretrained base models released in recent years. Our investigation reveals an exponential growth in maximum capability density over time, an empirical observation which we term the 'densing law', to characterize the rapid advancement in model efficiency. Through evaluation across five widely adopted benchmarks, we demonstrate that the maximum capability exhibits a doubling pattern approximately every 3.5 months. This remarkable progression suggests that within a 3.5-month period, an LLM with merely half of the parameters can achieve performance levels comparable to current state-of-the-art models. Furthermore, our formulation of the densing law yields several valuable corollaries about LLM training and applications.

Results

Densing law

To capture the development trend of LLM efficiency, we analysed 51 widely used open-source pretrained base models since the release of Llama-1 (ref. 4), including LLMs with various architectures such as vanilla transformer, sparse mixture-of-experts (MoE) and quantized models. We present the density of each model along with their release dates in Fig. 1. From the figure, we can observe that the density of LLMs has rapidly increased over time. Notably, the density of Llama-1, released in February 2023, is below 0.1, whereas more recently released models such as Gemma-2-9B and MiniCPM-3-4B have densities nearly reaching 2. To further illustrate the growth trend of the density of LLMs, we performed a linear fit on the envelope line in Fig. 1.

The maximum capability density of LLMs exhibits an exponential growth trend over time.

$$\ln(\rho_{\max}) = A \cdot t + B$$

Here, t is the time interval (unit: days) since the release date of Llama-1, $\rho_{\max}$ is the maximum density value at time t, and A and B are the parameters to be fitted.

We fitted these parameters based on our evaluation of five widely used benchmarks, MMLU[25], BBH[26], MATH[27], HumanEval[28] and MBPP[29]. Specifically, we obtained a growth coefficient $A \approx 0.007$, which implies that the density of LLMs doubles approximately every $\frac{\ln(2)}{A} \approx 3.5$ months. Here the R^2 for the linear regression function is 0.934.

From the rapid increase in model density, we can find the following. (1) This increase in density is largely attributed to the growth in the scale of pretraining data and improvements in the quality of the data. For example, Llama-1 is pretrained on 1.4 trillion tokens, whereas Llama-3 uses 15 trillion tokens with careful data cleaning. (2) Better performance does not always lead to better density. Llama-3.1-405B is currently one of the state-of-the-art open-source models owing to its large-scale parameter size. However, it is not the model with the highest density. This is because, constrained by computational resources and the scale of pretraining data, we usually cannot fully optimize the training settings for extremely large models, making them suboptimal in terms of cost-effectiveness.

The growth trend in model density reveals an important pattern in the development of current LLMs. Although scaling laws indicate that model performance improves with an increase in parameter size, the growth in parameter size is constrained by the limited computational resources available in deployment scenarios and the demand for fast response. As a result, large models are not simply evolving towards larger parameter sizes. Instead, developers of LLMs are striving for higher cost-effectiveness, aiming to achieve optimal performance with minimal inference costs. This discovery aligns with the principles discovered by Moore's law in the development of integrated circuit chips[30], which emphasize increasing transistor density on a limited chip area.

Capability density evaluation on a contamination-free dataset. Considering that the evaluation datasets used in the above sections are all public datasets, there exists a risk of data contamination. Therefore, to further validate the effectiveness of the densing law, we employed a contamination-free dataset, MMLU-CF[31], for evaluation. MMLU-CF is a dataset constructed in December 2024 that incorporates well-designed rewriting operations, ensuring that the questions in the dataset have never appeared in any corpus, thereby achieving contamination-free status. We follow the same method to process this dataset and compute the capability density of open-source models. The results are shown in Fig. 2. From the experimental results, we observe that the capability density measured using contamination-free data still exhibits an exponential growth trend. Here the R^2 for the linear regression function is 0.953. This finding is consistent with our discovery in the previous paragraphs. Meanwhile, the fitted growth coefficient A shows only approximately 1.5% difference (0.0065 versus 0.0066) with the main results from the previous paragraphs. This demonstrates the stability and generalizability of our findings.

Corollaries of the densing law

Based on the conclusion that the density of LLMs is continually increasing with an exponential trend, we provide the following corollaries and related observations.

The parameter sizes of LLMs with equivalent performance decrease exponentially. The densing law indicates that the capability density, which is defined as the ratio of the effective parameter size to the actual parameter size, increases exponentially. However, under the constraint of equivalent performance, the effective parameter size remains constant. Therefore, we can infer that the number of parameters required to achieve similar performance decreases exponentially over time. Specifically, the parameter size of LLMs with equivalent performance is halved approximately every 3.5 months. For example, MiniCPM-1-2.4B released on 1 February 2024, can achieve comparable or even superior performance with Mistral-7B released on 27 September 2023. This implies that, after just four months, an LLM with only 35% parameters can achieve roughly equivalent performance.

Inference costs for LLMs with equivalent performance decrease exponentially. Under the condition of achieving the equivalent performance, the actual parameter size of LLMs will decrease exponentially. This reduction in actual parameter count contributes to decreased computational costs during inference. Therefore, the exponential increase in the density of LLMs will directly result in an exponential decrease in inference costs for those models reaching the same performance level. To better illustrate the decreasing trend in the inference costs of LLMs, in Fig. 3a we present the application programming interface (API) pricing of LLMs that have achieved superior performance to GPT-3.5 since its release. From this figure, we can observe that the API pricing of LLMs also exhibits an exponential decline. Specifically, in December 2022, GPT-3.5 cost US$20 for one million tokens, whereas

https://doi.org/10.1038/s42256-025-01137-0

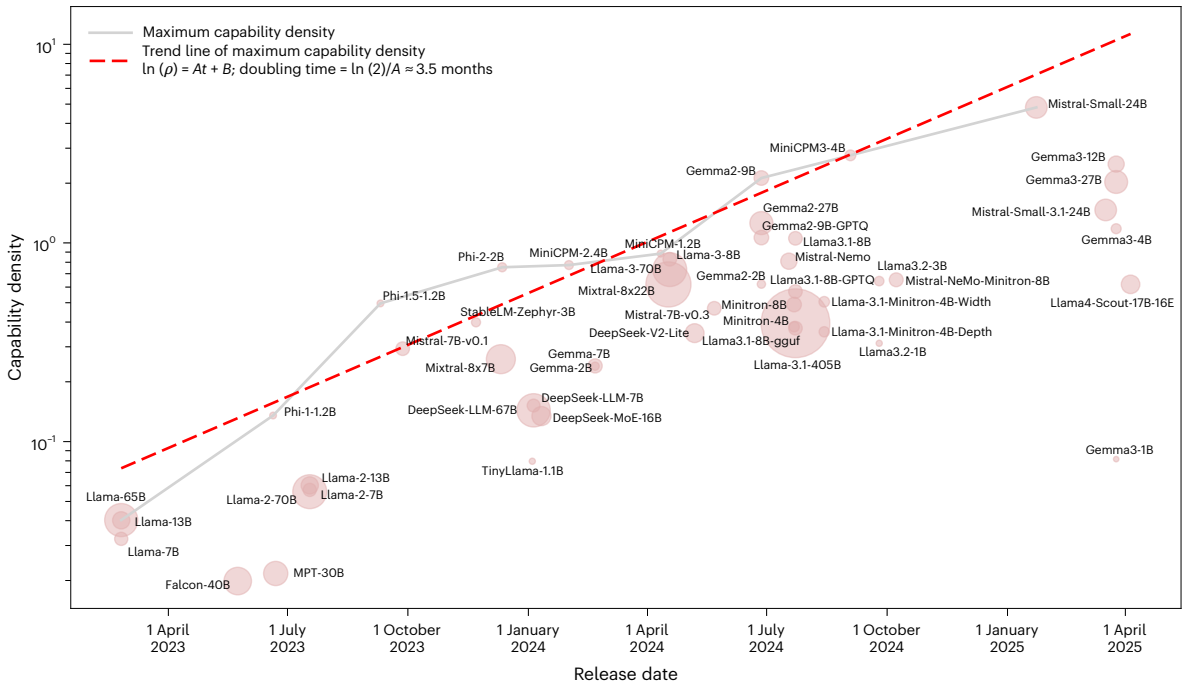


Fig. 1 | The estimated capability density of open-source base LLMs. We calculated capability density based on the evaluation results on five tasks, including MMLU for knowledge-intensive reasoning, BBH for logical reasoning, MATH for mathematical reasoning, and HumanEval and MBPP for coding. The area of each circle is proportional to the parameter size. We fitted a trend line between the release date and maximum density, and we found an empirical observation, which we call the densing law: the maximum capacity density grows exponentially over time.

by August 2024, Gemini-1.5-Flash cost only US$0.075 for the same number of tokens, which is a reduction by a factor of 266.7. Roughly speaking, the inference costs for LLMs halve approximately every 2.6 months. The exponentially decreasing trend of LLM API pricing is also observed in ref. 32.

In addition, we observe that the rate of decline in inference costs is faster than the growth rate of the density of LLMs. This is because inference costs depend not only on the actual parameter size but also heavily on the inference infrastructure. In recent years, inference systems for LLMs have garnered widespread attention from researchers, including optimizations in memory access speed for self-attention layers[33–35] and sparse computation optimizations for feed-forward networks[36–38]. These advancements have contributed greatly to the reduction in inference costs for LLMs.

'Densing law × Moore's law' reveals the great potential of edge-side intelligence. The densing law describes the exponential trend of increasing model density over time, focusing on improvements at the algorithmic level of LLMs. On the other hand, Moore's law, which states that computing power increases exponentially, highlights advancements in hardware technologies[30]. The combination of these two principles suggests a rapidly approaching future where high-quality LLMs can run efficiently on consumer-grade edge devices, such as smartphones and PCs, with low power consumption. This convergence of algorithmic efficiency and hardware capability is paving the way for more accessible and widespread use of advanced AI technologies in everyday devices.

Specifically, recent observations[39] show that the computing power of chips with the same price doubles approximately every 2.1 years. The densing law indicates that the ratio between the effective parameter size and the actual parameter size doubles every three months. Therefore, given a fixed chip price, the effective parameter size of the largest LLM that can run on the fixed-price chips grows exponentially. This growth rate is the product of the growth rate of the model density and the growth rate of the transistor density on the chip. Based on current estimates, this implies that the maximum effective parameter size approximately doubles every 88 days. This rapid growth highlights the combined impact of advancements in both algorithmic efficiency and hardware technologies, suggesting a future where increasingly powerful models can be deployed on existing hardware much more quickly than previously anticipated.

Efficient compression cannot always lead to density improvement. LLMs are often constrained by high inference costs, making it challenging to run them on consumer devices[40–43]. To address this issue, many developers employ pruning and distillation techniques to compress LLMs. These techniques are often believed to improve the performance of the resulting compressed models. In Fig. 3b, we present the densities of several models with their compressed counterparts. For example, Llama-3.2-3B/1B and Llama-3.1-minitron-4B[44] are derived from pruning and distilling Llama-3.1-8B[11], whereas Gemma-2-9B/2B is distilled from Gemma-2-27B[45]. In addition, we also employ a typical quantization method, GPTQ[46], to Gemma-2-9B and Llama-3.1-8B.

The results show that only the Gemma-2-9B model has a higher density than the original model, whereas all other compressed models have lower densities compared with their original counterparts. Intuitively, pruning involves removing unimportant neurons from LLMs, which suggests that these neurons might store less knowledge than other neurons. This would imply that compressed models should intuitively achieve higher density. However, the results are quite the opposite. This discrepancy might be due to the insufficient training of smaller models during the compression process, preventing them from reaching optimal density. In addition, quantization is a widely adopted method to reduce the memory costs of LLMs. The results show that the quantization will harm the model performance and lead to suboptimal capability density. Therefore, we encourage the community to address this challenge by ensuring that compressed models are adequately trained.

Density growth accelerated after ChatGPT's release. ChatGPT achieves great performance improvements across various tasks, and its zero-shot generalization ability spurs intensive efforts from both industry and academia to advance the development of LLMs. To illustrate the change in the trend of model density growth before and after

https://doi.org/10.1038/s42256-025-01137-0

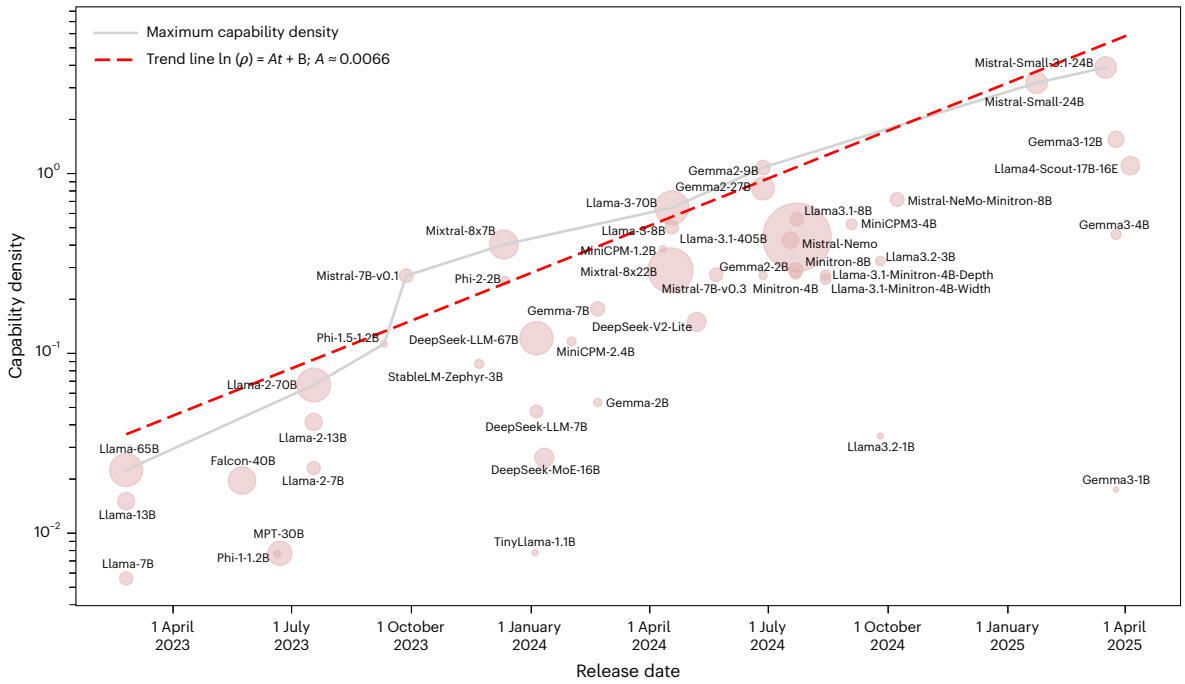


Fig. 2 | The estimated capability density on a contamination-free dataset, MMLU-CF. The area of each circle is proportional to the parameter size. The maximum capability density grows exponentially over time.

the release of ChatGPT, we evaluated the densities of typical LLMs since the release of GPT-3. We used the MMLU benchmark to capture the changes in density. The results are presented in Fig. 3c.

From the figure, we observe that the rate of increase in model density markedly accelerated following the release of ChatGPT. Before ChatGPT, the slope of the trend line was approximately $A \approx 0.0048$, whereas, after its release, it increased to $A \approx 0.0073$, indicating a 50% faster growth rate in model density. Several factors contribute to this accelerated growth. (1) Increased investment: the success of ChatGPT highlights the potential of LLMs, leading to a sharp increase in investment directed towards LLM development. (2) More high-quality open-source models: the rise in high-quality open-source models has lowered the barriers to research and development in LLMs. After ChatGPT's release, there was a notable increase in high-quality small LLMs with only billions of parameters, whose accessibility allowed many researchers to conduct LLM research using relatively small graphics processing unit (GPU) clusters. Therefore, we encourage the community to open-source their cutting-edge algorithms and models, which can meaningfully contribute to density improvement.

Density-optimal training is the pathway to achieving sustainable scaling laws. Since the release of GPT-3 (ref. 9) and the introduction of the scaling law[6], many researchers have focused on training LLMs with extremely large parameter sizes to continually enhance model performance. Guided by this trend, PaLM 540B[10], Gopher-280B[47] and Llama-3-405B[11] achieve great improvements on various natural language processing tasks. As capability density continues to grow, for every state-of-the-art LLM, a new model with comparable performance and lower inference costs will be available in a few months. This implies that each model has a relatively short period of optimal cost-effectiveness. Therefore, merely scaling model parameters and training data to enhance performance is neither sustainable nor cost-effective. Such an approach would lead to development costs that substantially outweigh the returns generated during the models' limited window of peak efficiency.

In light of the discovery of the densing law, we now encourage a shift towards density-optimal LLM pretraining. LLM developers need to shift their focus from optimizing performance by merely increasing training computation to optimizing density by technological innovations across multiple dimensions, including efficient model architectures, advanced training algorithms and sophisticated data preprocessing techniques. In addition, we should attach importance to the synergistic relationship between large and small models: large models can facilitate the development of high-density small models through knowledge distillation and data synthesis, whereas small models can enhance large-model training efficiency through data preprocessing, architecture experiments and weak-to-strong training techniques. By fostering this collaborative ecosystem and emphasizing technological innovations over simple resource scaling, density-optimal training aims to achieve the best results with optimal computational efficiency, thereby realizing a more sustainable and environmentally friendly scaling law that avoids excessive cost investments and the losses associated with short profit recovery cycles.

Discussion

Although the densing law reveals a clear exponential trend in LLM capability density, this observation raises several questions about the sustainability and mechanisms underlying this growth. In this section, we first discuss several important aspects and implications of the densing law and then introduce some future directions.

Accurate capability measurement

Capability density reflects the abilities of an LLM per unit of parameters. However, with current technologies, we cannot accurately assess the absolute capability level of LLMs, which means that quantifying intelligence remains a great challenge. Therefore, in this work, we designed a method to measure the relative density value of LLMs. In addition, we employed widely used benchmarks to evaluate the performance of LLMs. However, the limited number of benchmarks and potential data contamination issues introduced bias in performance evaluation. Thus, advancing more accurate measurement of the capabilities of LLMs or their intelligence levels in the future will enable better calculation of their densities.

Connection between the densing law and the scaling law

The scaling law of LLMs reveals the relationship between an LLM's performance and its parameter and data sizes, reflecting the intrinsic

https://doi.org/10.1038/s42256-025-01137-0

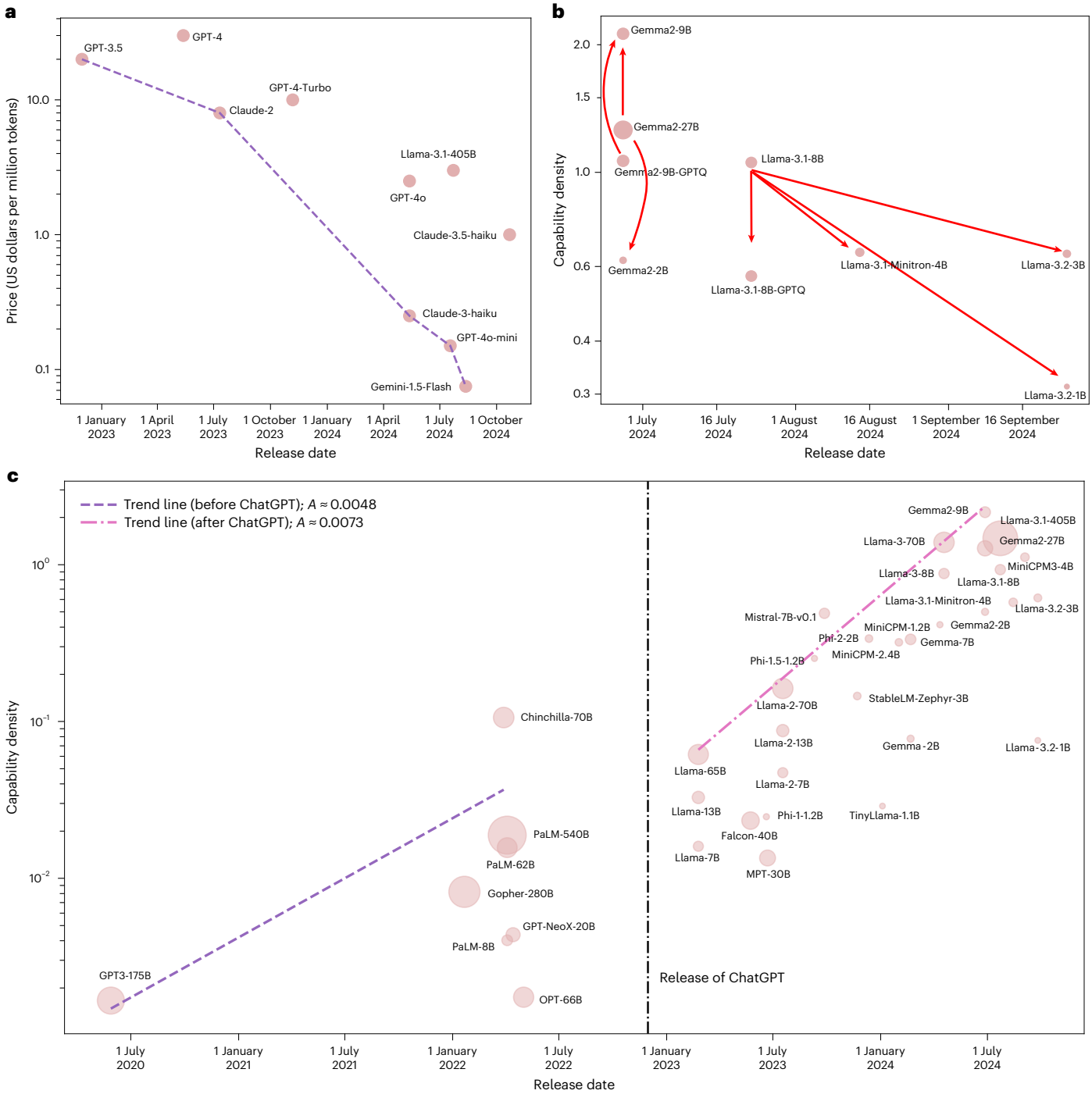


Fig. 3 | Observations for the corollaries of the densing law. a, API prices of LLMs that can outperform GPT-3.5. The line connects the cheapest models. The API prices of cheapest LLMs exhibit an exponential decline. **b**, Comparison between compressed models and their larger counterparts. This shows that efficient compression cannot always lead to density improvement. **c**, Density evaluated using MMLU. Two trend lines represent the growth of the density of LLMs before and after the release of ChatGPT. The rate of increase in model density markedly accelerated following the release of ChatGPT. The area of each circle in **b** and **c** is proportional to the parameter size.

characteristics of complex systems composed of vast numbers of neurons. The densing law further highlights the trend in the development of the efficiency and effectiveness of LLMs over time, marking a technological advancement trend as humanity pursues high-level AI models. Formally, under conditions of sufficient training data, the scaling law explains the relationship between model loss $\mathcal{L}$ and parameter size N as $\mathcal{L} = AN^{-\alpha}$, which is appropriate for the training of all transformer-based models. Here, A and α are hyper-parameters that need to be fitted. Furthermore, the densing law indicates that developers of LLMs can increase α through continual improvements in data, algorithms and architecture, thereby reducing the model loss for a given parameter size.

Key technologies for density improvement

The performance of LLMs depends on three key factors: model architecture, training algorithms and training data. Since the release of ChatGPT at the end of 2022, most open-source models have relied on the vanilla transformer architecture[48], with only minor modifications to activation

functions, normalization methods and similar components. Training algorithms have similarly remained largely unchanged, predominantly following the next token prediction paradigm. Therefore, it can be argued that the improvement in model capability density over the past two years has primarily been driven by the expansion of training data scale and enhancement of data quality.

Since early 2025, inspired by sparse (MoE) architectures[38,49,50], many developers have begun to experiment with more efficient designs that can achieve comparable performance with reduced computational requirements. Over the next one to two years, efficient model architectures will play an increasingly critical role in advancing model density improvements. However, the importance of training algorithms is gaining recognition. Reinforcement learning (RL) has been proved to effectively unlock model potential[51]; however, current RL applications are mostly limited to posttraining phases. Using RL to enhance foundation model performance during pretraining will become an important direction moving forward[52].

In summary, unlike the development of circuit density in semiconductors, which relies heavily on improving the etching finesse of central processing units (CPUs), the advancement of LLM capability density requires iterative innovation and optimizations across model architecture, training algorithms and training data.

Period of validity of the densing law

The densing law reveals the rapid development of LLM algorithms. In this paragraph, we discuss the question of how long this exponential growth in model density will continue. We believe that the rapid increase in model density is driven by massive investments in personnel and resources. The improvement of general intelligence capabilities in LLMs can bring substantial benefits to various industries, further encouraging investment in model research and development. Given the great potential of LLMs, we believe that the densing law will remain effective for a considerable period. However, it is essential to continually update the evaluation datasets used to evaluate model density, as LLMs will soon achieve satisfactory performance on existing datasets. In the event of achieving artificial general intelligence, LLMs themselves may be capable of conducting scientific research autonomously, exploring pathways to further increase density. At that point, the growth in LLM density could accelerate even more, driven by the abilities of the models to innovate and optimize their own development processes.

However, the capability density of LLMs cannot increase indefinitely. Similarly to how chip etching finesse is constrained by quantum tunnelling phenomena, the capability density of LLMs also faces theoretical limits. From an information-theoretic perspective, the amount of information that each parameter can store has a theoretical upper bound, which will ultimately lead to a deceleration in the growth rate of the densing law and eventual saturation. According to information theory principles[53], each floating-point parameter can only encode a finite number of bits at a given precision, and the compression of knowledge and capabilities faces insurmountable limits, which is analogous to how lossless compression algorithms cannot compress data below its entropy.

As we approach this theoretical limit, we may observe the following phenomena. First, density improvements achieved solely through algorithmic optimization and architectural enhancements will become increasingly challenging, with diminishing marginal returns. Then the industry may need to explore different computational paradigms, such as quantum computing, neuromorphic computing, or other breakthrough technologies, to transcend the density bottleneck inherent in classical computing architectures. This developmental trajectory mirrors the evolution of Moore's law—in the face of physical limits, technological progress will gradually transition from exponential to linear growth, until paradigm-shifting innovations are required to continue advancement.

Inference of the densing law

Recent research has highlighted that more inference computational costs allow LLMs to engage in deeper reasoning, effectively enhancing their performance on complex tasks[21]. In this work, we focus on density evaluation on the pretrained base models, which can serve as a good basis for further research for inference density. It is worth noting that the capabilities of base models largely determine the upper bounds of posttraining effectiveness. Existing research[54] has shown that the capability ceiling of LLMs after reinforcement learning is often constrained by the underlying base model's capabilities. Thus, understanding and modelling the development patterns of base model capability density is not only a crucial scientific question but also provides important theoretical value and practical guidance for subsequent reinforcement learning research. In the future, as research on posttraining scaling laws deepens, we can similarly further conduct research on inference densing laws, which we leave for future work.

Through our comprehensive analysis of LLM capability density, we provide an empirical observation, the densing law, which demonstrates that the capability density of LLMs increases exponentially over time, doubling approximately every 3.5 months. This empirical observation reveals the remarkable efficiency improvements in LLM development. The densing law complements traditional scaling laws by focusing on temporal efficiency trends. We hope that this finding will encourage the LLM community to continue enhancing capability density, achieving optimal performance with minimal computational costs and promoting more sustainable AI development.

Applications of the densing law

The densing law reveals the trend toward more efficient LLMs, with capability density serving as an evaluation metric for the cost-effectiveness of LLM usage, helping both developers and users to set reasonable goals and select appropriate models. Here we discuss several potential applications of the densing law.

In LLM technology development, the densing law provides LLM developers with optimization directions and evaluation metrics. Developers can establish scientific technical roadmaps based on the trend of capability density doubling every 3.5 months, avoiding the blind pursuit of parameter scale and instead focusing on improving parameter efficiency. This enables teams to allocate research and development resources more rationally and make forward-looking technical decisions based on trend predictions. In addition, the densing law provides a theoretical foundation for establishing unified model evaluation metrics. The introduction of capability density metrics enables fair comparisons across different models and scales, helping to establish dynamic performance benchmark systems and driving the entire AI industry towards more sustainable and efficient development. A prime example is DeepSeek-V3 (ref. 49), which diverges from the conventional parameter scaling paradigm adopted by open-source models such as Llama[11]. Instead, DeepSeek-V3 emphasizes efficient architectural design and achieves superior performance with substantially reduced computational overhead.

From a commercial applications perspective, the densing law provides an important reference for enterprise cost estimation of LLMs. When planning AI projects, enterprises can predict future usage costs based on capability density growth trends. For example, if a project requires a specific level of model performance, enterprises can calculate the current cost of achieving this performance and then predict cost changes in the coming months based on the 3.5-month doubling pattern.

In terms of edge hardware design, the development trend indicated by the densing law provides important guidance for hardware manufacturers. As capability density doubles every 3.5 months, future higher-level LLMs will be able to run on more economical chips, enabling efficient local AI inference in resource-constrained scenarios such as mobile devices and Internet-of-Things devices. For example,

models that currently require high-end GPUs may achieve the same or even better performance on mid-to-low-end chips in the future, lowering the hardware cost barrier for edge AI applications and promoting the widespread adoption of artificial intelligence technology across a broader range of devices and scenarios.

Methods

In this section, we formally define the density for LLMs, which is calculated as the ratio of the effective parameter size to the actual parameter size. In the following sections, we first describe the overall framework and formal definition of LLM density. Then we introduce how to use the scaling law to estimate the effective parameter size.

Overall framework and definition

The core of LLM density lies in the effective parameter size, which refers to the number of parameters required for a reference model to achieve the same performance as a given model. To achieve this, we need to fit a function that relates the parameter sizes of the reference model to its performance. Specifically, for a given model $\mathcal{M}$ with $N_{\mathcal{M}}$ parameters, assume that its performance score on the downstream tasks is $S_{\mathcal{M}}$. This score can be calculated using various metrics depending on the downstream task, such as accuracy and F1 score. To compute the effective parameter size, we trained a series of reference models with varying scales of parameters and training data. Based on these models, we fitted a function between the parameter size and performance: $S = f(N)$, where S denotes the downstream performance and N represents the parameter sizes of the reference model. Then we calculated the effective parameter size as $\hat{N}(S) = f^{-1}(S)$ and the density for $\mathcal{M}$ is defined as

$$\rho(\mathcal{M}) = \frac{\hat{N}(S_{\mathcal{M}})}{N_{\mathcal{M}}} = \frac{f^{-1}(S_{\mathcal{M}})}{N_{\mathcal{M}}}. \quad (1)$$

It is important to note that scaling laws are typically used to fit the relationship between language modelling loss and parameter sizes[6], and it is non-trivial to predict downstream task performance directly. Inspired by Llama-3 (ref. 11), we adopted a two-step estimation approach. (1) Loss estimation: in the first step, we used a series of reference models to fit the relationship between the parameter size and language modelling loss on the test set, expressed as $\mathcal{L} = f_1(N)$. (2) Performance estimation: owing to the presence of emergent abilities[55], it is challenging to accurately estimate the relationship between parameter sizes and performance using reference models with limited training computes. Therefore, we incorporated open-source models to compute their loss and performance on the test set and fitted the relationship $s = f_2(\mathcal{L})$. This two-step estimation process allowed us to derive $s = f_2(f_1(N))$. In the following sections, we will provide a detailed description of the fitting processes for $f_1(\cdot)$ and $f_2(\cdot)$.

Loss estimation

To predict the performance of downstream tasks, the first step involved fitting a function between the parameter size and language model loss using the scaling law widely adopted for LLM pretraining. Previous scaling laws primarily focus on language modelling loss on the whole sequences, which reflects the model's ability to estimate the probability of a given corpus. However, instances in the downstream tasks usually encompass both input instructions and output answers and we are primarily concerned with the probability of the output answers. Therefore, in this work, we focus on fitting the conditional loss $\mathcal{L} = -\log(P(\text{answer}|\text{instruction}))$. Concretely, we estimate a power-law function between the conditional loss $\mathcal{L}$ and parameter size N, as well as the number of training tokens D,

$$\mathcal{L} = aN^{-\alpha} + bD^{-\beta}, \quad (2)$$

where a, α, b and β are parameters that need to be fitted.

In previous research on scaling laws[6], the loss typically needs to be specified on a validation corpus, and the average loss is calculated over all tokens in this corpus. In this work, our goal is to fit the model's performance on downstream tasks, which require models to output the answers based on the input instructions. Therefore, we directly calculated the conditional loss on downstream tasks, meaning the loss incurred by the model when generating answers given the task inputs. (1) For multiple-choice problems, calculating the loss solely based on the content of the correct option can lead to inaccurate estimates, as it ignores the content of incorrect options. In addition, if we only calculated the loss on the final option labels, the loss for a single token was also unstable. Therefore, we concatenated the problem and its multiple options as inputs, and the output was the analysis for the input problem as well as the final answer label. (2) For most complex problems, such as mathematical questions, we often require the model to generate a sequence of reasoning steps before providing the final answer. For these tasks, when calculating the loss, we included both the reasoning steps and the correct answer as the output to compute the model's loss. It is important to note that most datasets do not provide reasoning steps for each instance. For both types of tasks, we used GPT-4o[5] to generate reasoning steps for all test instances. These approaches allowed us to better estimate the model's performance by considering the specific requirements and formats of different tasks.

Performance estimation

In the second step, we needed to predict downstream task performance based on the loss on test sets. In the loss-estimation step, the scaling law models trained with limited training computation usually cannot achieve meaningful scores on downstream tasks, with most scaling law models performing only at the level of random guessing. Thus, it is impossible to predict the downstream performance with only these models. To address this issue, we incorporated well-trained open-source models for function fitting and calculated their loss and performance on the test set. Considering that the performance for most downstream tasks is bounded, we used a sigmoid function for fitting. The sigmoid function naturally maps all input values to the range of 0 to 1. In addition, when the loss is particularly large, the model's performance should approximate that of random guessing, and when the loss is particularly small, the model's performance should approach the upper bound. This characteristic aligns with the properties of the sigmoid function, which is very flat at both extremes of the curve. Specifically, we estimated the downstream performance with the function

$$S = \frac{c}{1 + e^{-\gamma(\mathcal{L} - l)}} + d, \quad (3)$$

where c, γ, l and d are parameters that need to be estimated.

Density

In the above sections, we introduced the methodology for establishing the relationship between LLM parameter size and downstream task performance. Based on this relationship, we can derive the required parameter size for the reference model given a target model performance, thereby enabling the calculation of the target model's capability density. In this section, we further elaborate on the computational methods for model capability density, encompassing both vanilla dense models and efficient sparse models.

Density for dense models

After fitting equations (2) and (3), given the performance $S_{\mathcal{M}}$ of a model $\mathcal{M}$, we can infer the effective parameter size by using the inverse functions of these equations. It is important to note that, in equation (2), the loss $\mathcal{L}$ is a bivariate function of both the parameter count N and the training data size D. Therefore, when calculating the effective

parameter size, it is necessary to specify a particular training data size D. Here, to calculate the effective parameter size, we defaultly used $D = D_0 = 5$ trillion tokens. Then the effective parameter size can be explained as the parameter size the reference model trained with D_0 tokens needs to achieve equivalent performance. Concretely, we can compute the effective parameter size as

$$\hat{\mathcal{L}}(S_{\mathcal{M}}) = l - \frac{1}{\gamma}\ln\left(\frac{c}{S_{\mathcal{M}} - d} - 1\right); \quad \hat{N}(S_{\mathcal{M}}) = \left(\frac{\hat{\mathcal{L}}(S_{\mathcal{M}}) - bD_0^{-\beta}}{a}\right)^{-\frac{1}{\alpha}}. \tag{4}$$

Now we have established the relationship between the downstream performance and effective parameter size. The density of the given model $\mathcal{M}$ is $\rho(\mathcal{M}) = \frac{\hat{N}(S_{\mathcal{M}})}{N_{\mathcal{M}}}$. Intuitively, if one model can achieve better performance with the same scale of parameters, then the model's density is higher. Therefore, in the future, considering the limited computational resources of deployment devices, we should devote great effort to improving the model's density instead of merely increasing the model parameter sizes for better performance.

Density for efficient architecture

In the previous sections, we described the capability density of transformer-based LLMs, defining it as the ratio between the parameter size of a reference model with equivalent capability and the parameter size of the target model. Furthermore, for most users, parameter size is an imperceptible metric, and users are more concerned with the model's inference time. In addition, for some model variants, such as sparse MoE, the relationship between the parameter sizes and the computational costs is non-trivial, leading to inaccurate density computation. Therefore, in this section, we introduce a variant of capability density, using inference time as the density basis.

During model inference, time consumption mainly consists of two components: memory access time and computation time. For vanilla dense models, the model parameter size exhibits a roughly linear relationship with both memory access and computational inference time: that is, $T_{\text{mem}} = k_1 N$, and $T_{\text{comp}} = k_2 N$, where k_1 is determined by the chip's bandwidth and k_2 is determined by the chip's computational capability. Therefore, the above definition of model capability density can be equivalently expressed as follows: capability density is the ratio between the computational cost of a reference model with equivalent capability and the computational cost of the target model,

$$\rho = \frac{\hat{N}(S_{\mathcal{M}})}{N_{\mathcal{M}}} = \frac{\hat{N}(S_{\mathcal{M}})(k_1 + k_2)}{N_{\mathcal{M}}(k_1 + k_2)} = \frac{\hat{T}_{\text{mem}} + \hat{T}_{\text{comp}}}{T_{\text{mem}} + T_{\text{comp}}}. \tag{5}$$

However, for many efficient model algorithms, such as sparse MoE and model quantization, the relationship between model inference time cost and total parameter size becomes much more complex. Different methods reduce different portions of time overhead, and the distribution of time costs varies across different stages of the model. In the following, we discuss the time distribution of different efficient models.

Density for sparse MoE models. For sparse MoE models, a model contains both total parameters and activated parameters, with each token selecting different experts for computation. In addition, because the parameter size of MoE models often exceeds the storage capacity of a single end device chip, end-side MoE inference typically employs offloading strategies[16,56], which substantially increases memory access time during inference. Owing to the load-balancing characteristics among different experts in MoE models, most experts are used at least once, resulting in a linear relationship between the model's memory access time and the total parameter size. However, the model's computation time exhibits a linear relationship with the activated parameter size. We denote that, for an MoE model, $\mathcal{M}$, the total parameter size is $N_{\mathcal{M}}$ and the activated parameter size is $N^A_{\mathcal{M}}$. The capability density for the MoE model $\mathcal{M}$ is

$$\rho = \frac{\hat{N}(S_{\mathcal{M}})(k_1 + k_2)}{N_{\mathcal{M}} k_1^S + N^A_{\mathcal{M}} k_2}. \tag{6}$$

Here $k_1^S = k_1$ if the model can be fully loaded in the chip; otherwise $k^S \gg k_1$, because the bandwidth between the chip and disk/CPU memory is much lower than the chip's internal memory bandwidth.

Density for quantized models. As model parameter sizes continue to increase, using lower precision to store model parameters can effectively reduce model inference overhead[46,57,58]. Currently, widely applied quantization methods focus on quantizing model parameters while still employing half-precision computation during the computational process, thereby ensuring that model performance is not severely affected. Therefore, these quantization methods can only save the model's memory access time. Accordingly, the density of quantized models is defined as

$$\rho = \frac{\hat{N}(S_{\mathcal{M}})(k_1 + k_2)}{N_{\mathcal{M}} k_1^Q + N^A_{\mathcal{M}} k_2}. \tag{7}$$

Here, $k^Q < k_1$ and if the model is quantized into 4-bit, $k^Q = \frac{1}{4}k_1$.

To facilitate subsequent density computation, we assumed that the end devices use RTX 4090 chips, which have a theoretical FP16 computational capability of $C = 82.58$TFLOPS and a bandwidth of $B = 1.01$ TB s^{-1}. We assumed that the average input length for models is len = 100 tokens. Under these conditions, for an LLM without offloading techniques to use CPU memory, the computational time overhead during inference is approximately $\frac{2 \times N \times \text{len}}{C}$, and the memory access time overhead is approximately $\frac{N}{B}$. Therefore, we calculate that $k_1 \approx 2.45k_2$. We use this theoretical value in the density computations for MoE and quantized models. It is worth noting that, for MoE and quantized models, the total computation time can be further reduced through asynchronous memory access and computation, thereby reducing the total computation time of these efficient models and improving their corresponding capability density. This remains an open research question, so we use theoretical values to estimate capability density in this paper.

Using inference time as the computational basis for capability density allows the definition to be extended to models with more diverse architectures. However, this approach poses practical implementation challenges, as inference time is influenced by numerous factors, including hardware specifications (GPU type, memory and so on), testing scenarios (prefilling time versus decoding time) and software. Therefore, in diachronic analysis, for dense models, we still use parameter size as the basis for calculating capability density.

Evaluation settings

Here we introduce the datasets and models used in this paper.

Dataset. In this work, we adopted the following widely used datasets for evaluation: MMLU[25] for English knowledge-intensive tasks, BBH[26] for challenging logic reasoning tasks, MATH[27] for mathematical reasoning tasks, and HumanEval[28] and MBPP[29] for coding tasks. We applied the open-source tools from refs. 59,60 for evaluation. Here we evaluate all models in a few-shot in-context learning manner and these models were required to generate the final answer label based on the given demonstrations and inputs of test instances. Following widely used settings, MMLU, BBH, MATH, HumanEval and MBPP were evaluated under the 5-shot, 3-shot, 4-shot, 0-shot and 3-shot settings, respectively. In addition, for BBH, MATH and MBPP, we adopted the chain-of-thought prompting technique[61].

Table 1 | The detailed hyperparameters of small models trained for loss estimation

Name	Parameters	BS	n_{layer}	d	d_{ffn}	d_{head}	n_{head}	n_{kv}
0.005B	5,247,232	32	8	256	640	64	4	1
0.03B	31,470,080	32	12	512	1,280	64	8	2
0.1B	106,196,736	64	18	768	1,920	64	12	3
0.2B	245,416,960	128	24	1,024	2,560	64	16	2
0.4B	476,852,480	256	30	1,280	3,200	64	20	2
0.8B	828,225,024	512	36	1,536	3,840	64	24	3

Here BS refers to the training batch size. n_{layer}, d, d_{ffn}, d_{head}, n_{head} and n_{kv} represent number of layers, model hidden dimension, feedforward layer bottleneck dimension, attention head dimension, number of query heads and number of key-value heads, respectively.

Loss-estimation models. In the loss-estimation step, we needed to run a series of models with different scales of parameters and training data. These models are used as the reference models for further density computation. In this work, we adopted the training corpus of MiniCPM-3-4B[17], a widely used edge-size model, to train the small models. As for the model architecture, we used grouped query attention[62], gated feed-forward layers with SiLU as the activation function. We trained the models using a warm-up–stable–decay learning rate scheduler. To estimate the scaling curve, we trained the models with $\{10, 15, 20, 30, 40, 60\} \times N$ tokens, where N refers to the parameter size. We list the hyperparameters for small scaling models in Table 1.

Performance estimation models. In the performance estimation step, we introduced additional well-trained models to fit the loss-performance curve. Specifically, we used a series well-trained MiniCPM models and their intermediate training checkpoints. Notably, these models were trained with diverse recipes in the past two years. (1) Architectural diversity: covering different architectures such as vanilla self-attention and multihead latent attention. (2) Scale diversity: parameter sizes ranging from 0.36 B to 4 B. (3) Data diversity: each version used different training data. (4) Training method diversity: including both pretraining from scratch and continued pretraining based on existing models. We only required these models to share the same vocabulary as our scaling models.

Evaluated models. Furthermore, to illustrate the change in density over time, we selected widely used LLMs for evaluation since the release of Llama-1 (ref. 4), as most open-source models released before Llama-1 cannot achieve meaningful performance on our selected datasets. Specifically, we evaluated the density of the following models: Llama series of models[4,11,63], Falcon[64], MPT[65], Phi series of models[66–68], Mistral series of models[69], StableLM[70], TinyLlama[71], MiniCPM series of models[17], DeepSeek series of models[72] and Gemma series of models[45,73]. We prioritized using the results reported in each model's technical reports for density calculations. In addition, we only evaluated the density of base pretrained models without instruction tuning as the instruct-tuning datasets may have contained human-annotated data similar to our selected test data, leading to inaccurate density estimation. Notably, many pretrained models also introduce supervised fine-tuning datasets in the pretraining phase, leading to the test set contamination issue[74,75]. Thus, inaccurate density estimation remains to be solved, which we leave for future work.

Notably, we only evaluated the density of pretrained base models without further supervised fine-tuning and preference learning, owing to the following reasons. (1) Pretrained base models serve as the foundation for model performance. Considering the impact of further alignments, such as the quality of human annotations and the choice of alignment algorithms, introduces excessive confounding factors unrelated to the capabilities of the base model itself. (2) The scaling law for the performance of LLMs with alignment remains an open question that requires further exploration. Now there are numerous methods to improve the performance during inference time, such as retrieval-augmented generation[76], and thinking more about the inference scaling law[21]. Here we only consider the basic prompting technique for base LLM evaluation, as this technique cannot consistently improve the performance of this base model. We leave the density calculation for different inference FLOPs for future work, which may lead to an inference densing law.

Loss and performance estimation results

We present the estimation results of the two-step process in Extended Data Fig. 1. From the results, we observe that the two-step estimation process can effectively fit the performance of different-sized models on all downstream tasks. With the decrease in the loss on the test instances, the performance improves as a sigmoidal curve, and the loss has a power-law relationship with the number of parameters and training tokens.

To evaluate the effectiveness of our estimated method, we used models with less than four billion parameters to fit the loss-performance curve and preserved larger models for prediction. The triangles in Extended Data Fig. 1b are two models with tens of billions of parameters. From the results, we observe that we can effectively predict the downstream performance based on the loss values.

Reporting summary

Further information on research design is available in the Nature Portfolio Reporting Summary linked to this article.

Data availability

Datasets used for density estimation can be found at https://huggingface.co/datasets/openbmb/DensingLaw-ScalingBench. Models used for density estimation can be found at https://huggingface.co/openbmb/DensingLaw-ScalingModels.

Code availability

We employ OpenCompass (https://github.com/open-compass/opencompass)[59] for model evaluation and use Megatron-LM (https://github.com/NVIDIA/Megatron-LM)[77] for model training. Because models trained in this paper adhere to the standard decoder-only transformer architecture, the code from these repositories can be applied directly to reproduce our experimental results.

References

1. Bommasani, R. et al. On the opportunities and risks of foundation models. Preprint at https://arxiv.org/abs/2108.07258 (2021).
2. Qiu, X. et al. Pre-trained models for natural language processing: a survey. *Sci. China Technol. Sci.* **63**, 1872–1897 (2020).
3. Han, X. et al. Pre-trained models: past, present and future. *AI Open* **2**, 225–250 (2021).
4. Touvron, H. et al. Llama: open and efficient foundation language models. Preprint at https://arxiv.org/abs/2302.13971 (2023).
5. OpenAI et al. GPT-4 technical report. Preprint at https://arxiv.org/abs/2303.08774 (2023).
6. Kaplan, J. et al. Scaling laws for neural language models. Preprint at https://arxiv.org/abs/2001.08361 (2020).
7. Henighan, T. et al. Scaling laws for autoregressive generative modeling. Preprint at https://arxiv.org/abs/2010.14701 (2020).
8. Hoffmann, J. et al. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems* (eds Mohamed, S. et al.) 30016–30030 (NeurIPS, 2022).
9. Brown, T. et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems* (eds Larochelle, H. et al.) 1877–1901 (NeurIPS, 2020).
10. Chowdhery, A. et al. Palm: scaling language modeling with pathways. *J. Mach. Learn. Res.* **24**, 1–113 (2023).

11. Dubey, A. et al. The llama 3 herd of models. Preprint at https://arxiv.org/abs/2407.21783 (2024).
12. Sardana, N., Portes, J., Doubov, S. & Frankle, J. Beyond Chinchilla-optimal: accounting for inference in language model scaling laws. In *International Conference on Machine Learning* (eds Salakhutdinov, R. et al.) 43445–43460 (PMLR, 2024).
13. Yun, L., Zhuang, Y., Fu, Y., Xing, E. P. & Zhang, H. Toward inference-optimal mixture-of-expert large language models. Preprint at https://arxiv.org/abs/2404.02852 (2024).
14. OpenAI GPT-4o mini: advancing cost-efficient intelligence. *OpenAI* https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/ (2024).
15. Gunter, T. et al. Apple intelligence foundation language models. Preprint at https://arxiv.org/abs/2407.21075 (2024).
16. Xue, Z. et al. Powerinfer-2: fast large language model inference on a smartphone. Preprint at https://arxiv.org/abs/2406.06282 (2024).
17. Hu, S. et al. MiniCPM: unveiling the potential of small language models with scalable training strategies. In *First Conference on Language Modeling* (eds Choi, Y. et al.) (COLM, 2024).
18. Yao, Y. et al. Efficient gpt-4v level multimodal large language model for deployment on edge devices. *Nat. Commun.* **16**, 5509 (2025).
19. MiniCPM Team et al. Minicpm4: ultra-efficient LLMs on end devices. Preprint at https://arxiv.org/abs/2506.07900 (2025).
20. Brown, B. C. A. et al. Large language monkeys: scaling inference compute with repeated sampling. Preprint at https://arxiv.org/abs/2407.21787 (2024).
21. OpenAI Learning to reason with LLMs. *OpenAI* https://openai.com/index/learning-to-reason-with-llms/ (2024).
22. Snell, C., Lee, J., Xu, K. & Kumar, A. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. Preprint at https://arxiv.org/abs/2408.03314 (2024).
23. OpenAI OpenAI o1 system card. Preprint at https://arxiv.org/abs/2412.16720 (2024).
24. Google Gemini 2.5 Flash. *DeepMind* https://deepmind.google/models/gemini/flash/ (2024).
25. Hendrycks, D. et al. Measuring massive multitask language understanding. In *International Conference on Learning Representations* (eds Mohamed, S. et al.) (ICLR, 2021).
26. Suzgun, M. et al. Challenging big-bench tasks and whether chain-of-thought can solve them. In *Findings of the Association for Computational Linguistics* (eds Rogers, A. et al.) 13003–13051 (ACL, 2023).
27. Hendrycks, D. et al. Measuring mathematical problem solving with the MATH dataset. In *Neural Information Processing Systems Track on Datasets and Benchmarks* (eds Vanschoren, J. & Yeung, S.) (NeurIPS, 2021).
28. Chen, M. et al. Evaluating large language models trained on code. Preprint at https://arxiv.org/abs/2107.03374 (2021).
29. Austin, J. et al. Program synthesis with large language models. Preprint at https://arxiv.org/abs/2108.07732 (2021).
30. Moore, G. E. Cramming more components onto integrated circuits. *Electronics* **38**, 114–117 (1965).
31. Zhao, Q. et al. MMLU-CF: a contamination-free multi-task language understanding benchmark. In *Annual Meeting of the Association for Computational Linguistics* (eds Che, W. et al.) 13371–13391 (ACL, 2025).
32. Appenzeller, G. Welcome to LLMflation—LLM inference cost is going down fast. *a16z* https://a16z.com/llmflation-llm-inference-cost/ (2024).
33. Kwon, W. et al. Efficient memory management for large language model serving with pagedattention. In *Symposium on Operating Systems Principles* (eds Flinn, J. et al.) 611–626 (ACM, 2023).
34. Dao, T., Fu, D. Y., Ermon, S., Rudra, A. & Ré, C. Flashattention: fast and memory-efficient exact attention with io-awareness. In *Advances in Neural Information Processing Systems* (eds Koyejo, S. et al.) 16344–16359 (NeurIPS, 2022).
35. Dao, T. Flashattention-2: faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations* (eds Kim, B. et al.) (ICLR, 2024).
36. Song, Y., Mi, Z., Xie, H. & Chen, H. Powerinfer: fast large language model serving with a consumer-grade GPU. In *Symposium on Operating Systems Principles* (eds Witchel, E. et al.) 590–606 (ACM, 2024).
37. Liu, Z. et al. Deja vu: contextual sparsity for efficient LLMs at inference time. In *International Conference on Machine Learning* (eds Krause, A. et al.) 22137–22176 (PMLR, 2023).
38. Xiao, C. et al. Configurable foundation models: building LLMs from a modular perspective. Preprint at https://arxiv.org/abs/2409.02877 (2024).
39. Hobbhahn, M., Heim, L. & Aydos, G. Trends in machine learning hardware. *Epoch AI* https://epoch.ai/blog/trends-in-machine-learning-hardware (2023).
40. Ma, X., Fang, G. & Wang, X. LLM-Pruner: on the structural pruning of large language models. In *Advances in Neural Information Processing Systems* (eds Oh, A. et al.) 21702–21720 (NeurIPS, 2023).
41. Sun, M., Liu, Z., Bair, A. & Kolter, J. Z. A simple and effective pruning approach for large language models. In *International Conference on Learning Representations* (eds Kim, B. et al.) (ICLR, 2024).
42. Yang, C. et al. Survey on knowledge distillation for large language models: methods, evaluation, and application. *ACM Trans. Intell. Syst. Technol.* https://doi.org/10.1145/3699518 (2024).
43. Xu, X. et al. A survey on knowledge distillation of large language models. Preprint at https://arxiv.org/abs/2402.13116 (2024).
44. Muralidharan, S. et al. Compact language models via pruning and knowledge distillation. In *Advances in Neural Information Processing Systems* (eds Globersons, A. et al.) 41076–41102 (NeurIPS, 2024).
45. Team, G. et al. Gemma 2: improving open language models at a practical size. Preprint at https://arxiv.org/abs/2408.00118 (2024).
46. Frantar, E., Ashkboos, S., Hoefler, T. & Alistarh, D. OPTQ: accurate quantization for generative pre-trained transformers. In *International Conference on Learning Representations* (eds Liu, Y. et al.) (ICLR, 2023).
47. Rae, J. W. et al. Scaling language models: methods, analysis & insights from training gopher. Preprint at https://arxiv.org/abs/2112.11446 (2021).
48. Vaswani, A. et al. Attention is all you need. In *Advances in Neural Information Processing Systems* (eds Guyon, I. et al.) 5998–6008 (NeurIPS, 2017).
49. Liu, A. et al. Deepseek-v3 technical report. Preprint at https://arxiv.org/abs/2412.19437 (2024).
50. Fedus, W., Zoph, B. & Shazeer, N. Switch transformers: scaling to trillion parameter models with simple and efficient sparsity. *J. Mach. Learn. Res.* **23**, 120 (2022).
51. Guo, D. et al. Deepseek-r1: incentivizing reasoning capability in LLMs via reinforcement learning. *Nature* **645**, 633–638 (2025).
52. Zelikman, E. et al. Quiet-star: language models can teach themselves to think before speaking. In *First Conference on Language Modeling* (eds Choi, Y. et al.) (COLM, 2024).
53. Shannon, C. E. A mathematical theory of communication. *Bell Syst. Tech. J.* **27**, 379–423 (1948).
54. Yue, Y. et al. Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model? Preprint at https://arxiv.org/abs/2504.13837 (2025).

55. Wei, J. et al. Emergent abilities of large language models. *Trans. Mach. Learn. Res.* (2022).
56. KVCache.ai KTransformers: a flexible framework for experiencing cutting-edge LLM inference optimizations. *GitHub* https://github.com/kvcache-ai/ktransformers (2024).
57. Dettmers, T., Pagnoni, A., Holtzman, A. & Zettlemoyer, L. QLORA: efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems* (eds Oh, A. et al.) 10088–10115 (NeurIPS, 2023).
58. Xiao, G. et al. SmoothQuant: accurate and efficient post-training quantization for large language models. In *International Conference on Machine Learning* (eds Krause, A. et al.) 38087–38099 (PMLR, 2023).
59. OpenCompass OpenCompass: a universal evaluation platform for foundation models. *GitHub* https://github.com/open-compass/opencompass (2023).
60. Liu, J., Xia, C. S., Wang, Y. & Zhang, L. Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems* (eds Oh, A. et al.) 21558–21572 (NeurIPS, 2023).
61. Wei, J. et al. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems* (eds Koyejo, S. et al.) 24824–24837 (NeurIPS, 2022).
62. Ainslie, J. et al. GQA: training generalized multi-query transformer models from multi-head checkpoints. In *Conference on Empirical Methods in Natural Language Processing* (eds Bouamor, H. et al.) 4895–4901 (ACL, 2023).
63. Touvron, H. et al. Llama 2: open foundation and fine-tuned chat models. Preprint at https://arxiv.org/abs/2307.09288 (2023).
64. Almazrouei, E. et al. The falcon series of open language models. Preprint at https://arxiv.org/abs/2311.16867 (2023).
65. Team, M. N. Introducing MPT-30B: raising the bar for open-source foundation models. *Databricks* https://www.databricks.com/blog/mpt-30b (2023).
66. Gunasekar, S. et al. Textbooks are all you need. Preprint at https://arxiv.org/abs/2306.11644 (2023).
67. Li, Y. et al. Textbooks are all you need ii: phi-1.5 technical report. Preprint at https://arxiv.org/abs/2309.05463 (2023).
68. Abdin, M. et al. Phi-3 technical report: a highly capable language model locally on your phone. Preprint at https://arxiv.org/abs/2404.14219 (2024).
69. Jiang, A. Q. et al. Mistral 7B. Preprint at https://arxiv.org/abs/2310.06825 (2023).
70. Bellagente, M. et al. Stable LM 2 1.6B technical report. Preprint at https://arxiv.org/abs/2402.17834 (2024).
71. Zhang, P., Zeng, G., Wang, T. & Lu, W. Tinyllama: an open-source small language model. Preprint at https://arxiv.org/abs/2401.02385 (2024).
72. Liu, A. et al. Deepseek-v2: a strong, economical, and efficient mixture-of-experts language model. Preprint at https://arxiv.org/abs/2405.04434 (2024).
73. Team, G. et al. Gemma: open models based on Gemini research and technology. Preprint at https://arxiv.org/abs/2403.08295 (2024).
74. Wei, T. et al. Skywork: a more open bilingual foundation model. Preprint at https://arxiv.org/abs/2310.19341 (2023).
75. Dominguez-Olmedo, R., Dorner, F. E. & Hardt, M. Training on the test task confounds evaluation and emergence. In *International Conference on Learning Representations* (eds Yue, Y. et al.) (ICLR, 2025).
76. Lewis, P. et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems* (eds Larochelle, H. et al.) 9459–9474 (2020).
77. Shoeybi, M. et al. Megatron-LM: training multi-billion parameter language models using model parallelism. *GitHub* https://github.com/NVIDIA/Megatron-LM (2019).

Acknowledgements

This work was supported by the National Key R&D Program of China (grant no. 2022ZD0116312), Beijing Municipal Science and Technology Plan Project (grant no. Z241100001324025) and a grant from the Guoqiang Institute, Tsinghua University. We thank AI9Stars and the OpenBMB open-source community for their support. C.X. is supported by the China National Postdoctoral Program for Innovative Talents (grant no. BX20250388) and the Shuimu Tsinghua Scholar Program.

Author contributions

C.X., X.H. and Z.L. conceived and initiated this research project. The conceptual framework of capability density was collaboratively developed through discussions between C.X., X.H., G.Z., Z.L. and M.S. C.X. drafted the initial manuscript. X.H. revised and refined the manuscript. The experimental implementation was carried out by J.C., W.Z. and B.L., who developed the codebase and conducted comprehensive model evaluations. Dataset collection for scaling law estimation was performed by J.Z., Z.Z., B.L. and J.C. The final manuscript underwent thorough proofreading and review by X.H., Z.L. and M.S. to ensure accuracy and clarity.

Competing interests

The authors declare no competing interests.

Additional information

Extended data is available for this paper at https://doi.org/10.1038/s42256-025-01137-0.

Supplementary information The online version contains supplementary material available at https://doi.org/10.1038/s42256-025-01137-0.

Correspondence and requests for materials should be addressed to Xu Han, Zhiyuan Liu or Maosong Sun.

Peer review information *Nature Machine Intelligence* thanks Yue Cheng and the other, anonymous, reviewer(s) for their contribution to the peer review of this work.

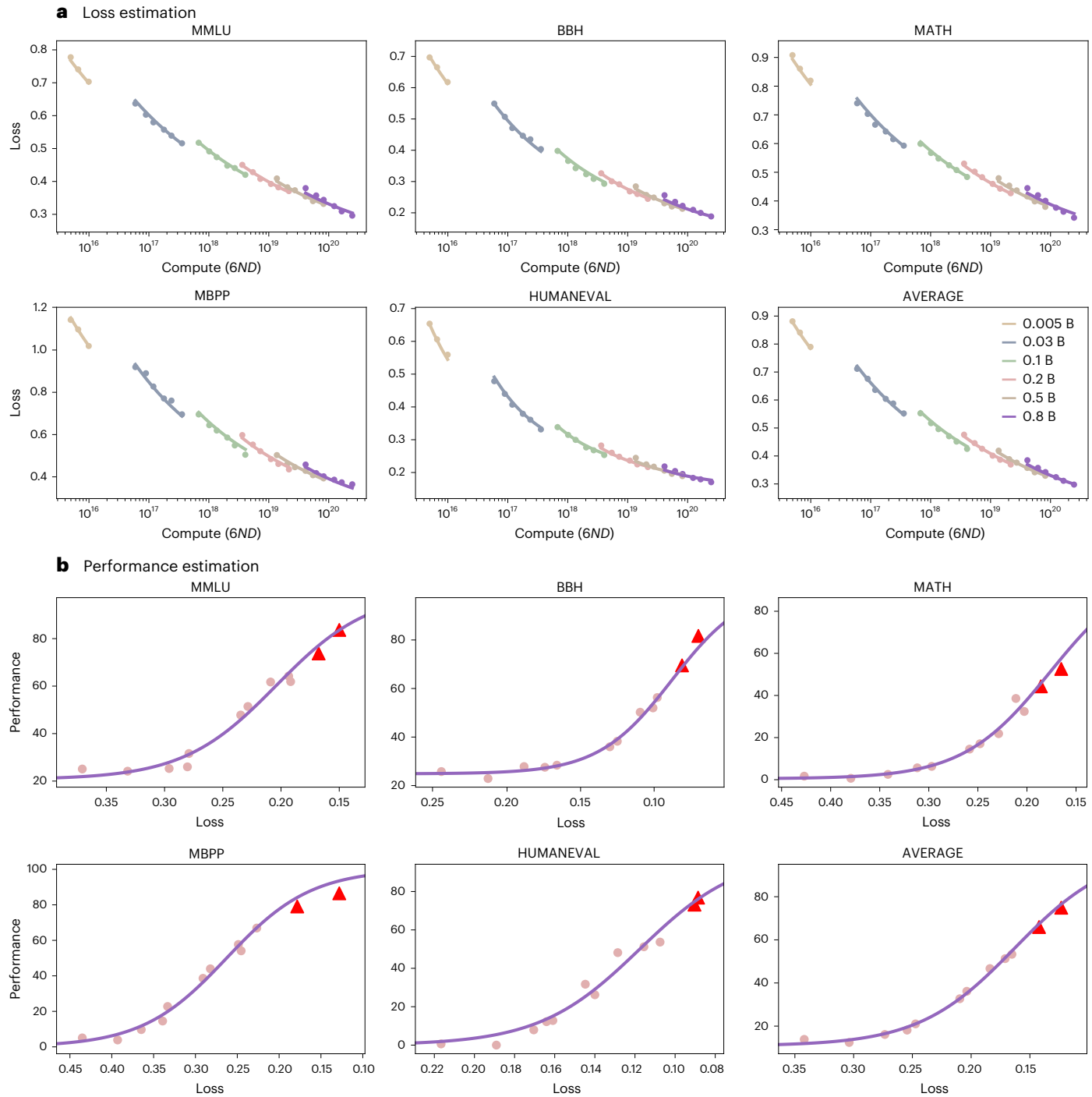
Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

© The Author(s) 2025

https://doi.org/10.1038/s42256-025-01137-0



Extended Data Fig. 1 | The results for loss estimation and performance estimation. The lines are fitted curves. **a**, The *x* axis refers to the pretraining computation, which is approximated by Compute = 6*ND*. **b**, Triangles are larger models for prediction.

nature portfolio

Corresponding author(s): Xu Han, Zhiyuan Liu, Maosong Sun

Last updated by author(s): Sep 9, 2025

Reporting Summary

Nature Portfolio wishes to improve the reproducibility of the work that we publish. This form provides structure for consistency and transparency in reporting. For further information on Nature Portfolio policies, see our Editorial Policies and the Editorial Policy Checklist.

Statistics

For all statistical analyses, confirm that the following items are present in the figure legend, table legend, main text, or Methods section.

n/a	Confirmed	
☐	☒	The exact sample size (*n*) for each experimental group/condition, given as a discrete number and unit of measurement
☒	☐	A statement on whether measurements were taken from distinct samples or whether the same sample was measured repeatedly
☐	☒	The statistical test(s) used AND whether they are one- or two-sided *Only common tests should be described solely by name; describe more complex techniques in the Methods section.*
☒	☐	A description of all covariates tested
☒	☐	A description of any assumptions or corrections, such as tests of normality and adjustment for multiple comparisons
☒	☐	A full description of the statistical parameters including central tendency (e.g. means) or other basic estimates (e.g. regression coefficient) AND variation (e.g. standard deviation) or associated estimates of uncertainty (e.g. confidence intervals)
☒	☐	For null hypothesis testing, the test statistic (e.g. *F*, *t*, *r*) with confidence intervals, effect sizes, degrees of freedom and *P* value noted *Give P values as exact values whenever suitable.*
☒	☐	For Bayesian analysis, information on the choice of priors and Markov chain Monte Carlo settings
☒	☐	For hierarchical and complex designs, identification of the appropriate level for tests and full reporting of outcomes
☐	☒	Estimates of effect sizes (e.g. Cohen's *d*, Pearson's *r*), indicating how they were calculated

Our web collection on statistics for biologists contains articles on many of the points above.

Software and code

Policy information about availability of computer code

Data collection	We employ OpenCompass~(\url{https://github.com/open-compass/opencompass})~\citep{opencompass} for model evaluation and utilize Megatron-LM~(\url{https://github.com/NVIDIA/Megatron-LM})~\citep{megatron-lm} for model training. Since models trained in this paper adhere to the standard decoder-only Transformer architecture, the code from these repositories can be directly applied to reproduce our experimental results.
Data analysis	After obtaining the model's performance on downstream tasks, we calculated the model density according to the method definition in the paper, which was performed in Excel.

For manuscripts utilizing custom algorithms or software that are central to the research but not yet described in published literature, software must be made available to editors and reviewers. We strongly encourage code deposition in a community repository (e.g. GitHub). See the Nature Portfolio guidelines for submitting code & software for further information.

Data

Policy information about availability of data

All manuscripts must include a data availability statement. This statement should provide the following information, where applicable:

- Accession codes, unique identifiers, or web links for publicly available datasets
- A description of any restrictions on data availability
- For clinical datasets or third party data, please ensure that the statement adheres to our policy

Datasets used for density estimation can be found at \url{https://huggingface.co/datasets/openbmb/DensingLaw-ScalingBench}. Models used for density estimation can be found at \url{https://huggingface.co/openbmb/DensingLaw-ScalingModels}.

Research involving human participants, their data, or biological material

Policy information about studies with human participants or human data. See also policy information about sex, gender (identity/presentation), and sexual orientation and race, ethnicity and racism.

Reporting on sex and gender	Our research does not involve human participants.
Reporting on race, ethnicity, or other socially relevant groupings	Our research does not involve human participants.
Population characteristics	Our research does not involve human participants.
Recruitment	Our research does not involve human participants.
Ethics oversight	Our research does not involve human participants.

Note that full information on the approval of the study protocol must also be provided in the manuscript.

Field-specific reporting

Please select the one below that is the best fit for your research. If you are not sure, read the appropriate sections before making your selection.

☐ Life sciences ☐ Behavioural & social sciences ☒ Ecological, evolutionary & environmental sciences

For a reference copy of the document with all sections, see nature.com/documents/nr-reporting-summary-flat.pdf

Ecological, evolutionary & environmental sciences study design

All studies must disclose on these points even when the disclosure is negative.

Study description	We introduce the concept of capability density as a metric to evaluate the quality of the LLMs and describe the trend of LLMs in terms of both effectiveness and efficiency. The capability density is calculated based on the downstream performance and parameter size of these LLMs.
Research sample	We use widely-used datasets to evaluate the model performance and adopt widely-used LLMs for evaluation.
Sampling strategy	N/A
Data collection	We download the widely-used LLMs from their open-source repositories and evaluat the models using OpenCompass to collect the performance of these LLMs on downstream tasks.
Timing and spatial scale	We choose LLMs which are published in the past two years since the release of ChatGPT.
Data exclusions	No data are excluded.
Reproducibility	The readers can directly employ Megatron-LM to train a series of reference models and use OpenCompass to evaluate the performance of other open-source LLMs. Then following the definition of capability density of LLMs, the readers can compute the capability densities of given LLMs.
Randomization	N/A
Blinding	All models evaluated in our paper are open-source models. Blinding is not suitable for our research.

Did the study involve field work? ☐ Yes ☒ No

Reporting for specific materials, systems and methods

We require information from authors about some types of materials, experimental systems and methods used in many studies. Here, indicate whether each material, system or method listed is relevant to your study. If you are not sure if a list item applies to your research, read the appropriate section before selecting a response.

Materials & experimental systems

n/a	Involved in the study
☒	☐ Antibodies
☒	☐ Eukaryotic cell lines
☒	☐ Palaeontology and archaeology
☒	☐ Animals and other organisms
☒	☐ Clinical data
☒	☐ Dual use research of concern
☒	☐ Plants

Methods

n/a	Involved in the study
☒	☐ ChIP-seq
☒	☐ Flow cytometry
☒	☐ MRI-based neuroimaging

Plants

Seed stocks	N/A
Novel plant genotypes	N/A
Authentication	N/A